

Bin Packing — Why It's Hard, How to Approximate It, and When It Isn't Hard At All

Priyanshu Debnath ·

priyanshudebnath.me/blog/bin-packing-np-hardness-approximation-algorithms/

Published 2026-07-07

A full pass through bin packing — the formal problem, why the decision version is NP-complete (and why the optimization version is strongly so), every classical approximation algorithm with its proven bound, the special cases that collapse back to polynomial time, and the backtracking/bitmask-DP techniques you'd actually use to solve small instances in a contest.

I was solving [Codeforces 2038L — Bridge Renovation](#), got a clean closed-form answer out of it, and then asked myself the obvious follow-up: what if the constraints hadn't been so generous? That question is what turned into this post — the formal bin-packing problem, exactly why it's NP-hard (and the sharper, less commonly explained fact that it's *strongly* NP-hard), every classical approximation algorithm with a proof of its worst-case bound (not just a citation of one), the handful of special cases where the problem quietly becomes polynomial, and the backtracking/bitmask techniques you'd reach for if a contest handed you a small instance and expected an exact answer.

1 A concrete starting point: Codeforces 2038L — Bridge Renovation

The problem. Monocarp needs to renovate three bridges of widths 18, 21, and 25 units — he needs n planks of length 18, n planks of length 21, and n planks of length 25 (the same n for all three). Planks are only sold in a standard length of 60, workers can cut a plank into pieces but refuse to join two planks together, and offcuts are just waste. Given $1 \leq n \leq 1000$, find the minimum number of standard planks. For $n = 1$: you can't fit all three required lengths in one plank ($18 + 21 + 25 = 64 > 60$), so the answer is 2 — one plank cut into 25 and 18 (17 wasted), the other into 21 (39 wasted).

Solving it. First, enumerate every way to cut a single 60-length plank into pieces of length 18, 21, and 25 — there are only 13 such patterns, since the numbers are so constrained (at most three 18s, two 21s, or two 25s fit in one plank at all). Sorted by how much of the plank they use:

Pattern (18s, 21s, 25s)	Length used	Waste
(1, 2, 0) — 18 + 21 + 21	60	0
(2, 1, 0) — 18 + 18 + 21	57	3
(3, 0, 0) — 18 + 18 + 18	54	6
(0, 0, 2) — 25 + 25	50	10
(0, 1, 1) — 21 + 25	46	14
(1, 0, 1) — 18 + 25	43	17
... and 7 more, all strictly worse		

Three patterns do all the real work: (1, 2, 0) wastes nothing, (3, 0, 0) is the most efficient pure source of 18s, and (0, 0, 2) is the only way to get 25s without dragging in an 18 or 21 you don't need yet. The construction: cut $\lfloor n/2 \rfloor$ planks as (1, 2, 0) — this exactly satisfies the 21-requirement (or comes up one short, if n is odd) and produces $\lfloor n/2 \rfloor$ of the needed 18s, leaving $\lceil n/2 \rceil$ still to get. Cut $\lceil n/2 \rceil$ planks as (0, 0, 2) to cover the 25s — that's already $\lfloor n/2 \rfloor + \lceil n/2 \rceil = n$ planks

total before touching the leftover 18s at all. Cover the remaining $\lceil n/2 \rceil$ eighteens with $(3, 0, 0)$ planks (occasionally swapping one for a $(2, 1, 0)$ plank when n is odd, to pick up that last 21 for free instead of using an extra plank) — that's $\lceil \lceil n/2 \rceil / 3 \rceil$ more planks, which is exactly $\lceil n/6 \rceil$ by the standard nested-ceiling identity $\lceil \lceil n/a \rceil / b \rceil = \lceil n/(ab) \rceil$. Total:

$$\text{planks}(n) = n + \left\lceil \frac{n}{6} \right\rceil$$

I checked this construction by brute force against every n from 0 to 25 (exhaustive BFS over the true state space, no shortcuts) before trusting it — it matched exactly every time.

Proving you can't do better. A construction only tells you an upper bound; proving $n + \lceil n/6 \rceil$ is *optimal* needs a matching lower bound, and "total wood needed divided by plank length" isn't tight here ($\lceil 64n/60 \rceil \approx 1.067n$, well under the true $\approx 1.167n$) — the pieces don't divide evenly, so volume alone underestimates the real cost. The right tool is a weighting argument (this is LP duality, done by hand): assign a weight to each unit you need — $y_{18} = 1/3$, $y_{21} = 1/3$, $y_{25} = 1/2$ — chosen so that **every one of the 13 feasible patterns** (a, b, c) satisfies $a \cdot y_{18} + b \cdot y_{21} + c \cdot y_{25} \leq 1$. Check the three patterns doing the real work: $(1, 2, 0) \rightarrow \frac{1}{3} + \frac{2}{3} = 1$, $(3, 0, 0) \rightarrow 1$, $(0, 0, 2) \rightarrow 1$ — all tight, by design. Check the rest and every single one comes in at ≤ 1 too (I verified all 13 by hand; the worst of the "off" patterns, $(0, 1, 1)$, gives $\frac{1}{3} + \frac{1}{2} = \frac{5}{6}$).

Now take *any* valid solution using P planks, whatever patterns it uses. Since every pattern satisfies that inequality, summing it over all P planks gives $\frac{1}{3}(\text{total 18s}) + \frac{1}{3}(\text{total 21s}) + \frac{1}{2}(\text{total 25s}) \leq P$. The solution must produce at least n of each length, so the left side is at least $\frac{n}{3} + \frac{n}{3} + \frac{n}{2} = \frac{7n}{6}$. Therefore $P \geq \frac{7n}{6}$, and since P is an integer, $P \geq \lceil 7n/6 \rceil$. One more identity — $\lceil 7n/6 \rceil = \lceil n + n/6 \rceil = n + \lceil n/6 \rceil$ since n is already an integer — and this lower bound exactly matches the construction above. ■

The "what if" that leads to this post. That whole solution leaned on the constraints being generous in a very specific way: only *three* fixed, small item lengths, a fixed bin size, and both chosen so that a handful of hand-enumerable cutting patterns dominate everything else. That's what made "enumerate 13 patterns and solve a tiny weighted covering problem" work at all. Now strip those generousities away — arbitrary item sizes instead of three fixed constants, arbitrarily many distinct sizes instead of three, no clean divisibility relationship to bin capacity — and the number of "patterns" explodes combinatorially, the weighting trick stops finding a clean closed form, and you're standing at the edge of the general bin-packing problem. That's where the rest of this post picks up.

2 The problem, stated plainly

You're given n items with positive sizes $s_1, s_2, \dots, s_n$ and bins of uniform capacity C . Every item must go into exactly one bin, and no bin's contents may sum past C . The **optimization version** asks for the minimum number of bins. The **decision version** asks a yes/no question instead: can everything fit into at most k bins? That distinction matters more than it looks like it should — the complexity argument in the next section is built entirely on the decision version, and then extended to the optimization version afterward.

Two numbers come up constantly, so name them now: $S = \sum_i s_i$ is the total size of everything, and $\lceil S/C \rceil$ is the obvious *lower bound* on the number of bins — you can never do better than "total volume divided by bin volume," even though you can absolutely do worse.

3 Why it's hard: a full pass through the complexity argument

I'm not going to assume you remember the exact definitions here, because half the value of "explaining bin packing's complexity" is being precise about what these words mean rather than

gesturing at them.

P is the class of decision problems solvable in time polynomial in the input size. **NP** is the class of decision problems where a proposed "yes" answer can be *verified* in polynomial time, even if finding that answer might take much longer — for bin packing's decision version, the certificate is just the assignment of items to bins, and checking it means summing each bin once, which is $O(n)$.

NP-hard means every problem in NP reduces to it in polynomial time — informally, it's "at least as hard as anything in NP," but it doesn't have to be in NP itself (it might not even be a decision problem). **NP-complete** is the intersection: NP-hard *and* in NP. Bin packing's decision version is a textbook NP-complete problem. Its optimization version — "find the minimum k ," not just "is k enough" — is NP-hard but isn't itself a yes/no question, so "NP-complete" doesn't technically apply to it; showing the decision version is NP-complete is what makes the optimization version NP-hard (an oracle that solves the optimization version in poly time would let you binary-search k and solve the decision version in poly time too).

3.1 The reduction: PARTITION becomes a 2-bin question

The standard proof reduces from **PARTITION**: given a multiset of positive integers $A = \{a_1, \dots, a_m\}$ with total sum S , can A be split into two subsets with equal sum? **PARTITION** is a classical NP-complete problem (it's *weakly* NP-complete, a distinction that turns out to matter — more on that in a moment).

The reduction is almost embarrassingly direct: take the same multiset as your bin-packing items, set the bin capacity to $C = S/2$, and ask whether two bins suffice. If S is odd, no partition can exist and the answer is trivially "no" either way. If S is even, then packing everything into two bins of capacity $S/2$ is only possible if each bin's contents sum to *exactly* $S/2$ — any other split would push one bin over capacity, since the two bins together must account for all of S . So **PARTITION** on A is solvable exactly when bin packing on A fits into 2 bins of capacity $S/2$:

$$\text{PARTITION}(A) \iff \text{BinPack}(A, k=2, C=S/2)$$

That's a polynomial-time reduction (it's literally the identity on the input, plus one division), which means 2-bin decision packing is at least as hard as **PARTITION**. Since the certificate-checking argument above put it in NP, and this reduction puts it in NP-hard, bin packing's decision version is NP-complete. ■

3.2 The sharper fact: bin packing is *strongly* NP-hard

Here's the detail most casual treatments skip. **PARTITION** is only **weakly** NP-hard — it has a pseudo-polynomial time algorithm (the classic subset-sum DP, $O(nS)$), so if your numbers are small (polynomially bounded in n), **PARTITION** is easy in practice. The **PARTITION** reduction above only shows bin packing is *weakly* NP-hard too, for the specific case of $k = 2$.

The general bin-packing optimization problem is hard in a strictly stronger sense: it's **strongly** NP-hard, meaning no algorithm can solve it in time polynomial in n *and* $\max_i s_i$ — not even if every item size is small. No pseudo-polynomial algorithm exists unless $P = NP$. This is proven by reducing from **3-PARTITION** instead: given $3m$ positive integers with total sum mT , where every single integer is constrained to lie strictly between $T/4$ and $T/2$, decide whether they can be grouped into m triples each summing to exactly T . That size constraint is the whole trick — it forces *exactly three* elements into every group in any valid solution (two elements can sum to at most just under T , and four elements sum to more than T at minimum), which is what makes **3-PARTITION** so much more rigid, and so much harder, than **PARTITION**. Garey and Johnson proved **3-PARTITION** strongly NP-complete in *Computers and Intractability* (1979), and

it remains the standard hammer for proving strong NP-hardness across bin packing, scheduling, and cutting-stock problems to this day.

Reducing to bin packing is the same trick as before — items are the $3m$ integers, capacity is T , ask if m bins suffice — except now the size restriction guarantees a feasible packing must use exactly three items per bin, which is exactly the structure 3-PARTITION needs. Because the reduction keeps the numbers polynomially bounded, this shows bin packing itself is strongly NP-hard, not just weakly so.

Why should you care about weak versus strong? Because it tells you where hope is allowed to live. A weakly NP-hard problem might have a fast exact algorithm hiding behind small numbers — subset sum, knapsack, PARTITION. Bin packing doesn't get that escape hatch. The exponential blowup is fundamental to the structure of the problem, not an artifact of large item sizes.

4 Exact algorithms

For small instances, or as a subroutine inside something bigger, you sometimes want the actual optimum rather than a bound on how far off you might be.

- **Brute force** — try every assignment of items to bins. Astronomically slow ($O(k^n)$ -ish), useful only as a correctness oracle for testing faster methods.
- **Bitmask DP** — see the competitive-programming section below; this is the one you'd actually reach for at $n \leq 20$.
- **ILP formulation** — binary variables $x_{ij} = 1$ if item i goes in bin j , and $y_j = 1$ if bin j is used at all; minimize $\sum_j y_j$ subject to $\sum_j x_{ij} = 1$ for every item and $\sum_i s_i x_{ij} \leq C \cdot y_j$ for every bin. Solvable with any off-the-shelf ILP solver for moderate sizes, though symmetry between interchangeable bins tends to slow branch-and-bound down badly unless you add symmetry-breaking constraints.
- **Branch and bound** — Martello and Toth's MTP algorithm and Korf's bin-completion algorithm are the two names that come up repeatedly in the literature; both exploit strong lower bounds and dominance rules to prune aggressively, and both can solve instances with hundreds of items in practice, far beyond what brute force or bitmask DP could touch.
- **The cutting-stock connection** — bin packing *is* the cutting-stock problem (minimizing the number of raw material rolls cut into ordered piece lengths), and the Gilmore–Gomory formulation solves its LP relaxation via column generation: instead of enumerating every possible "pattern" of items in a bin up front (there can be exponentially many), you generate promising patterns on demand by solving a knapsack subproblem. This is the actual industrial approach for large instances, and it routinely gets within a fraction of a bin of optimal.

5 Approximation algorithms

This is the heart of the problem in practice — nobody runs an ILP solver to pack VMs onto hosts every few seconds. Every algorithm below is greedy, runs in at most $O(n \log n)$, and comes with a *proven* worst-case guarantee relative to OPT, the optimal bin count.

5.1 Next Fit

Keep exactly one bin "open." If the current item fits, place it there; otherwise close that bin for good, open a fresh one, and place it there. Next Fit never looks back at an old bin, which is what makes it fast (genuinely $O(n)$, no data structure needed) and also what makes it wasteful.

An interactive visualization is available in the online version: <https://priyanshudebnath.me/blog/bin-packing-np-hardness-approximation-algorithms/>

Watch bin 1 in that trace: it closes with 4 units of slack the instant item 3 doesn't fit, and that slack is gone forever — Next Fit will never place anything there again, even though a later small item might have fit perfectly. Run First Fit on the exact same six items and it packs them into **3** bins instead of Next Fit's 4, precisely because it's willing to look back.

Claim: $\text{NF}(I) \leq 2 \cdot \text{OPT}(I)$.

Proof. Let bins $1, 2, \dots, k$ be the bins Next Fit uses, in the order it opens them. Take any two consecutive bins i and $i + 1$. Next Fit only opens bin $i + 1$ because the item it was about to place didn't fit in bin i — meaning at that moment, $\text{fill}(i) + \text{fill}(i + 1) > C$ (the item's size plus whatever was already in bin i exceeds C , and that item is now sitting in bin $i + 1$). Since bin i never receives anything after bin $i + 1$ opens — Next Fit only ever adds to the *current* last bin — that inequality holds for the final packing too, not just at that instant.

Pair up the bins: $(1, 2), (3, 4), \dots$. Each pair sums to more than C , so $\lfloor k/2 \rfloor$ pairs contribute more than $\lfloor k/2 \rfloor \cdot C$ in total item size. That total is at most $S = \sum_i s_i \leq \text{OPT}(I) \cdot C$ (since $\text{OPT}(I)$ bins suffice to hold everything). So $\lfloor k/2 \rfloor \cdot C < \text{OPT}(I) \cdot C$, giving $\lfloor k/2 \rfloor < \text{OPT}(I)$, i.e. $k < 2 \text{OPT}(I) + 1$. Since k and $\text{OPT}(I)$ are both integers, $k \leq 2 \text{OPT}(I)$. ■

This bound is also *tight* — there are instances where Next Fit genuinely needs arbitrarily close to twice the optimal bin count (alternate items of size just over $C/2$ with items of size just under $C/2$; Next Fit is forced to open a fresh bin for every single item, while an optimal packing pairs them up two per bin), so $2 \cdot \text{OPT}(I)$ isn't a loose worst-case artifact of this particular proof — it's the actual answer.

5.2 First Fit and Best Fit

First Fit scans every open bin in order and places the item in the first one it fits. **Best Fit** scans every open bin and places the item in whichever one leaves the *least* leftover space (breaking ties however you like). Both need a way to query "which bins fit this item" efficiently — a balanced BST keyed by remaining capacity gets both down to $O(n \log n)$.

An interactive visualization is available in the online version: <https://priyanshudebnath.me/blog/bin-packing-np-hardness-approximation-algorithms/>

An interactive visualization is available in the online version: <https://priyanshudebnath.me/blog/bin-packing-np-hardness-approximation-algorithms/>

That Best Fit example is worth sitting with for a second. Run First Fit on the identical six items and it *also* finishes in 3 bins — but with 2, 3, and 3 units of slack scattered across them. Best Fit finishes with 2, 6, and **0** — it drives one bin to exactly full. Same bin count, different local decisions, because Best Fit's criterion ("leave as little room as possible") concentrates leftover space instead of spreading it thin. That's a genuinely different greedy policy, even though the two algorithms are asymptotically just as good as each other in the worst case.

Claim: $\text{FF}(I) \leq 2 \cdot \text{OPT}(I)$ (and the identical proof works for Best Fit — see below).

Proof. Let bins $1, \dots, k$ be First Fit's bins, in the order opened. Take any two bins $i < j$. Bin j was opened at some moment because the item about to be placed — call it x — didn't fit in *any* of bins $1, \dots, j - 1$, in particular not in bin i . That means, at that moment, $\text{fill}(i) > C - \text{size}(x)$. Bin fills only grow over time, so this still holds for the final packing: $\text{fill}(i) > C - \text{size}(x)$. And x itself ended up in bin j , so $\text{fill}(j) \geq \text{size}(x)$. Adding these: $\text{fill}(i) + \text{fill}(j) > C$.

This is the *same* pairing argument as Next Fit's proof above — every earlier bin and every later bin sum to more than C — so it concludes identically: $k \leq 2 \text{OPT}(I)$. ■

Best Fit satisfies the same bound, by the same proof, because Best Fit only opens a new bin under the identical condition — the item doesn't fit any currently open bin — so the argument "bin i must have had insufficient room for x at the moment bin j opened" holds unchanged.

That's the complete, elementary proof of the classical factor-of-2 bound, and it's genuinely all you need for most purposes. The *tighter* $1.7 \cdot \text{OPT}(I)$ bound Johnson proved in 1973 is real and correct, but its proof is a different animal — it assigns each item a carefully calibrated *weight* (roughly, $w(s) \approx s/C$ but adjusted upward for items just over $C/6$, $C/3$, and similar thresholds, tuned so that no First-Fit bin can ever hold more than 1.7 units of total weight) and bounds the number of bins by the total weight instead of the total size directly. It's correct and it's the tight bound, but reproducing the full case analysis honestly takes several pages, not a paragraph — Johnson's original 1973 MIT thesis runs over 100 pages precisely because of proofs like this one. I'm citing it rather than re-deriving it here, and the $2 \times$ proof above is offered as the complete, self-contained version rather than a simplification that quietly loses rigor.

5.3 First Fit Decreasing (and Best Fit Decreasing)

Sort the items largest-first, then run First Fit. That's the entire algorithm, and it's the one I'd reach for by default if you told me "just pack these well and don't overthink it."

An interactive visualization is available in the online version: <https://priyanshudebnath.me/blog/bin-packing-np-hardness-approximation-algorithms/>

That example packs eight items summing to 36 into exactly 4 bins — which is optimal, since $\lceil 36/10 \rceil = 4$, and three of the four bins finish completely full.

FFD is First Fit run on one particular, carefully-chosen input order — so the $\text{FF}(I) \leq 2 \cdot \text{OPT}(I)$ proof above applies to it *for free*, with zero extra work: $\text{FFD}(I) \leq 2 \cdot \text{OPT}(I)$ is already proven. Sorting first buys something much stronger than that, though: Johnson's 1973 paper established the asymptotic ratio $11/9 \approx 1.222$ — a real improvement over the bare factor of 2, and the proof technique is the same weight-function idea sketched above, just calibrated further using the fact that the input is sorted (which rules out certain bad interleavings the unsorted proof has to defend against). It took until 2007 for Dósa to show that ratio is *tight*, and until 2013 for a complete published proof of the exact additive constant, $\text{FFD}(I) \leq \frac{11}{9}\text{OPT}(I) + \frac{6}{9}$ — that's four decades between "here's a good bound" and "here's the provably best-possible constant," which says something honest about how much harder pinning down the *exact* worst case is compared to a serviceable one. Best Fit Decreasing shares the identical $11/9$ asymptotic bound. In practice, on anything resembling a realistic size distribution, FFD tends to land within a bin or two of optimal — the $11/9$ figure is a worst-case guarantee, not a typical outcome.

5.4 Beyond simple greedy: Karmarkar–Karp

None of the algorithms above get *arbitrarily* close to optimal — their guarantees are constant multiplicative factors. Karmarkar and Karp's 1982 algorithm does dramatically better: it solves the LP relaxation of the Gilmore–Gomory cutting-stock formulation and rounds the fractional solution, achieving

$$\text{KK}(I) \leq \text{OPT}(I) + O(\log^2 \text{OPT}(I))$$

in polynomial time — an *additive* gap that grows only polylogarithmically, not a multiplicative one. This is what's called an asymptotic fully polynomial-time approximation scheme (AFPTAS); a simpler asymptotic PTAS (without the polylog refinement) was given earlier by de la Vega and Lueker in 1981. It's the closest thing to "solved" that a strongly NP-hard problem is ever going to get.

5.5 Online algorithms: the same problem, but you can't look ahead

Everything above assumes you see the whole item list up front. In the **online** setting, items arrive one at a time and you must place each irrevocably before seeing the next — this is the realistic model for a VM scheduler accepting requests as they come in, and it's a meaningfully harder setting: you can't sort, and you can't undo a bad early decision.

Result	Bound	Source
Any online algorithm	competitive ratio ≥ 1.54278	Balogh, Békési, Dósa, Epstein, Levin, 2021
Best known online algorithm (Advanced Harmonic)	competitive ratio ≤ 1.57829	Balogh et al., 2018
Classical Harmonic algorithm	competitive ratio $\rightarrow 1.691\dots$	Lee & Lee, 1985

That gap between 1.54278 and 1.57829 is still open — nobody knows the exact optimal online competitive ratio, which says something about how genuinely subtle the online version is even after four decades of people chipping away at both bounds.

6 Special and common cases

"NP-hard" describes the *general* problem. Plenty of instances you'll actually encounter are much easier, and it's worth knowing exactly which structural properties buy you that.

- **Equal-size items.** If every item has identical size s , the problem stops being combinatorial entirely: each bin holds exactly $\lfloor C/s \rfloor$ items, so the answer is simply $\left\lceil \frac{n}{\lfloor C/s \rfloor} \right\rceil$. *Proof:* no bin can ever hold more than $\lfloor C/s \rfloor$ items regardless of which items go where (that's just $s \cdot \lfloor C/s \rfloor \leq C < s \cdot (\lfloor C/s \rfloor + 1)$), and any bin count up to that per-bin maximum is trivially achievable since the items are interchangeable — so the bin count is forced to be exactly $\lceil n/\lfloor C/s \rfloor \rceil$, both as a lower bound (packing fewer bins can't hold n items at $\lfloor C/s \rfloor$ each) and as an upper bound (fill every bin to that limit, last one however many remain). A closed-form expression, computable in $O(1)$ after one division. The entire hardness of bin packing lives in the *variety* of item sizes, not in having many items.
- **Equal bin sizes vs. variable bin sizes.** Everything above assumes uniform capacity C . Variable-Sized Bin Packing (bins of several different capacities, choose freely) is a real generalization used in cloud placement (different instance-type sizes), and it inherits the same NP-hardness — but Murgolo (1987) gave an APTAS for it too, so asymptotically it's no worse than the uniform case.
- **Items of exactly half the bin capacity.** If every item is exactly $C/2$, pack two per bin — again trivial. But a *mix* of near- $C/2$ items and small items is precisely the structure behind the classical adversarial constructions used to prove First Fit's and Next Fit's tight bounds: interleave items just over $C/2$ (which can never share a bin with each other) with items that would fit neatly into the slack, but present them in an order that keeps the algorithm from ever pairing them up. That's not a special case that makes the problem *easier* — it's the special case that makes the worst-case bounds *tight*.
- **Everything fits in one bin.** If $\sum_i s_i \leq C$, the answer is trivially 1. Always worth checking first; it costs one linear pass.
- **Any single item exceeds C .** Infeasible outright — no packing exists, and this needs to be checked before running anything else, since every algorithm above silently assumes $s_i \leq C$ for all i .
- **Items that evenly divide C .** If every item size divides C , First Fit-style algorithms tend

to reach zero waste, because there's never a "left over but too small to matter" remainder — every bin either fills exactly or has room for another full-size chunk.

- **The 3-PARTITION structure.** Items strictly between $C/4$ and $C/2$, forced into groups of exactly three — this is the case that's *provably* as hard as the general problem gets (strong NP-hardness), not a special case at all in the sense of being easier, but worth knowing by sight, because if your instance happens to have this shape, you should stop looking for a clever polynomial algorithm and reach for backtracking or an ILP solver instead.

7 Competitive programming: exact algorithms for small instances

Contest constraints are usually kind enough that you're never actually asked to solve general bin packing — you're asked to solve it for $n \leq 20$ or so, where exponential-but-small-base algorithms are completely fine.

Bitmask DP. Let $dp[mask]$ be the minimum number of bins needed to pack exactly the item subset represented by $mask$. Precompute, for every subset, whether its total size fits in a single bin. Then

$$dp[mask] = 1 + \min_{\substack{s \subseteq mask \\ sum(s) \leq C}} dp[mask \setminus s]$$

Naively enumerating submasks of every mask is $O(3^n)$ (the standard "sum over subsets" bound), which is comfortably fast for $n \leq 18-20$.

Correctness. By induction on $|mask|$. Base case $dp[\emptyset] = 0$ is trivially correct — zero items need zero bins. For the inductive step, look at *any* optimal packing of the items in $mask$: it uses some number of bins, and in particular its *last* bin holds some subset $s \subseteq mask$ with $sum(s) \leq C$ (a valid pattern) — removing that bin leaves an optimal packing of $mask \setminus s$ using one fewer bin (if it weren't optimal for $mask \setminus s$, you could swap in a better one and improve the original packing too, contradiction). So $dp[mask] \leq 1 + dp[mask \setminus s]$ for that specific s , and since the transition minimizes over *every* feasible s , it's at least as good — giving $dp[mask] = 1 + \min_s dp[mask \setminus s]$ exactly. ■

```

1 int dp[1 << 20];
2 int fits[1 << 20]; // 1 if this subset's total size <= C
3
4 int solve(int n) {
5     int full = (1 << n) - 1;
6     fill(dp, dp + (1 << n), INT_MAX);
7     dp[0] = 0;
8     for (int mask = 1; mask <= full; mask++) {
9         for (int sub = mask; sub; sub = (sub - 1) & mask) {
10             if (fits[sub] && dp[mask ^ sub] != INT_MAX) {
11                 dp[mask] = min(dp[mask], dp[mask ^ sub] + 1);
12             }
13         }
14     }
15     return dp[full];
16 }
```

DFS with backtracking and pruning. This is the approach that actually scales past what bitmask DP can reach, and it's the one worth internalizing since the *pruning* is where all the real technique lives. Sort items largest-first (this alone helps prune earlier), then recursively try placing

each item into every currently open bin plus one fresh bin. Two prunings matter enormously in practice:

1. **Symmetry breaking.** All *empty* bins are interchangeable — trying an item in empty bin #3 after already trying it in empty bin #2 explores the same structural possibility twice. Never branch into more than one empty bin per item; this alone collapses a huge fraction of the redundant search tree.
2. **Lower-bound pruning.** At any point in the search, if $(\text{bins used so far}) + \left\lceil \frac{\text{remaining unplaced size}}{C} \right\rceil \geq (\text{best solution found so far})$, abandon the branch immediately — it can't possibly improve on what you already have.

Neither pruning can ever discard the true optimum, which is the property that actually matters here. Symmetry breaking is safe because any solution that places an item in empty bin #3 while empty bin #2 stays unused is identical, bin-count-wise, to the solution with the labels of bins #2 and #3 swapped — so refusing to explore "item goes in the *second* empty bin" never loses a distinct solution, only a relabeled duplicate of one you'll still reach. Lower-bound pruning is safe because $\lceil (\text{remaining size})/C \rceil$ is a valid lower bound on bins still needed no matter how the rest of the search plays out (it's the same volume argument from the very first section) — so if current-bins-plus-that-bound already meets or exceeds your best answer so far, no completion of this branch can beat it, by definition of "lower bound."

```

1  int best;
2
3  void dfs(int i, vector<int>& binRemaining, int itemsRemainingSum, int C) {
4      if ((int)binRemaining.size() >= best) return; // already worse than best
5      int lowerBound = binRemaining.size() + (itemsRemainingSum + C - 1) / C;
6      if (lowerBound >= best) return; // can't possibly win from here
7
8      if (i == (int)items.size()) {
9          best = min(best, (int)binRemaining.size());
10         return;
11     }
12
13     for (int b = 0; b < (int)binRemaining.size(); b++) {
14         if (binRemaining[b] >= items[i]) {
15             binRemaining[b] -= items[i];
16             dfs(i + 1, binRemaining, itemsRemainingSum - items[i], C);
17             binRemaining[b] += items[i];
18         }
19         if (binRemaining[b] == C) break; // symmetry break: only try ONE empty bin
20     }
21     // also try opening a brand new bin
22     binRemaining.push_back(C - items[i]);
23     dfs(i + 1, binRemaining, itemsRemainingSum - items[i], C);
24     binRemaining.pop_back();
25 }
```

For n up to roughly 30–40 — past what bitmask DP can hold in memory but still too small to trust a bound-and-prune search alone — **meet in the middle** is the other tool worth knowing: split the items into two halves, compute all achievable "bin configurations" for each half independently, then merge.

A couple of real problems to actually implement this against:

- [Codeforces Gym 102770B — Bin Packing Problem](#). The problem stated close to verbatim — good for getting First Fit correct as a baseline before you reach for the exact DFS version above.
- [Codeforces 1066D — Boxes Packing](#). Not framed as "bin packing" in the title, but it's a Next-Fit-style greedy packing simulation wrapped in a binary-search-on-the-answer outer loop — a good bridge between the online-algorithm theory above and an actual accepted submission.

If you want to sanity-check your own bitmask DP or backtracking implementation, the cleanest self-test is to run it against brute force on small random instances ($n \leq 10$) and confirm the bin counts always agree — it's a five-minute script and it'll catch off-by-one errors in the pruning bound immediately.

8 Variants, briefly

The 1D case above is the classical version, but the same "pack items into bounded containers" shape generalizes in a few directions worth knowing exist even if this post doesn't go deep on them:

- **Vector bin packing** — items and bins are d -dimensional vectors instead of scalars (think CPU *and* memory *and* disk simultaneously), which is the actual model behind real cloud VM placement, not the 1D version.
- **2D and 3D geometric packing** — rectangles or boxes instead of scalar sizes, with or without rotation allowed. Considerably harder to even approximate well than the 1D case.
- **Bin covering** — the dual problem: instead of minimizing bins while respecting a capacity, maximize the number of bins you can fill to at least some threshold. Same flavor of hardness, opposite objective.

9 Applications and which algorithm to actually reach for

Kubernetes' bin-packing scheduling strategy is, structurally, First Fit Decreasing on resource vectors — pack pods onto the fewest nodes, biggest resource requests first. Cutting stock in the paper and steel industries runs on Gilmore–Gomory column generation because the instances are large enough and the material cost high enough that a fraction-of-a-percent improvement over FFD is worth the engineering effort. Ad-slot scheduling and container loading both reduce to close variants of the same problem.

For anything you're implementing yourself: **First Fit Decreasing is the correct default**. It's a few lines of code, runs in $O(n \log n)$, and the 11/9 worst-case bound means you're never more than about 22% off optimal even in an adversarial instance — in practice it's usually far closer than that. Reach for Best Fit instead only if you specifically care about concentrating leftover space (bin-consolidation heuristics sometimes want this). Reach for an online algorithm only when you genuinely can't batch and sort — items truly do arrive one at a time and must be placed immediately. And reach for exact search — bitmask DP or the pruned DFS above — only when n is small enough that "provably optimal" is actually achievable in the time you have.

10 Summary

- Bin packing: pack n items of given sizes into the fewest bins of capacity C . Decision version (does k bins suffice) is NP-complete; the optimization version is NP-hard.
- The 2-bin case reduces from PARTITION (weakly NP-hard). The general case reduces

from 3-PARTITION and is **strongly** NP-hard — no pseudo-polynomial algorithm can exist unless $P = NP$.

- Next Fit: ratio 2. First Fit / Best Fit: ratio 1.7. First Fit Decreasing / Best Fit Decreasing: tight ratio $11/9 \cdot \text{OPT} + 6/9$. Karmarkar–Karp: $\text{OPT} + O(\log^2 \text{OPT})$, essentially optimal.
- Online bin packing is provably harder: best known algorithm is 1.57829-competitive, best known lower bound is 1.54278 — the exact answer is still an open problem.
- Equal-size items, single-bin-suffices, and evenly-divisible-size instances all collapse to easy special cases; the 3-PARTITION structure is exactly the case that stays maximally hard.
- For small n , bitmask DP ($O(3^n)$) or a properly pruned DFS (symmetry-broken, lower-bound-pruned) gets you the exact optimum — that pruning is the actual skill, not the recursion itself.

Resources – where to go deeper

Problems to actually solve

- [Codeforces 2038L — Bridge Renovation](#). The problem this whole post grew out of — a tightly constrained bin-packing instance (3 fixed item sizes) that has a clean closed form once you see the pattern-enumeration + weighting trick.
- [Codeforces Gym 102770B — Bin Packing Problem](#). The classical problem stated almost verbatim — good for implementing First Fit correctly as a baseline.
- [Codeforces 1066D — Boxes Packing](#). A Next-Fit-style greedy simulation wrapped in a binary-search outer loop.

Community discussion worth reading

- [Codeforces: "How to divide N integers into some groups such that the sum of each group doesn't exceed a value, minimizing group count?"](#) — this *is* bin packing, discussed from a purely contest-tactics angle.
- [Codeforces: "Bin Packing with Multiple bins and limitations on bin capacity"](#) — a worked discussion of the bitmask-DP approach for small n .
- [USACO Guide — Bitmask DP](#). Good general treatment of the subset-DP technique this post's bitmask solution relies on, beyond just bin packing.

Primary sources, if you want the actual proofs in full

- D. S. Johnson, *Near-Optimal Bin Packing Algorithms*, PhD thesis, MIT, 1973 — dspace.mit.edu/handle/1721.1/57819. The origin of the First Fit / First Fit Decreasing bounds, weight-function proof technique included, all 100+ pages of it.
- M. R. Garey and D. S. Johnson, *Computers and Intractability: A Guide to the Theory of NP-Completeness*, W. H. Freeman, 1979. The canonical reference for 3-PARTITION and strong NP-completeness generally — still the right place to actually learn this material properly, not just read a summary of it.
- Wikipedia's [Bin packing problem](#) and [3-partition problem](#) articles are solid, well-cited starting points if you want a map of the wider literature before diving into primary sources.

11 References

- D. S. Johnson. *Near-Optimal Bin Packing Algorithms*. PhD thesis, MIT, 1973. dspace.mit.edu/handle/1721.1/57819 — First Fit / Best Fit's $1.7 \cdot \text{OPT}$ bound and First Fit Decreasing's asymptotic $11/9$ ratio.

- M. R. Garey and D. S. Johnson. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman, 1979 — 3-PARTITION and strong NP-completeness.
- G. Dósa. *The Tight Bound of First Fit Decreasing Bin-Packing Algorithm Is $\text{FFD}(I) \leq \frac{11}{9}\text{OPT}(I) + \frac{6}{9}$* . 2007. link.springer.com/chapter/10.1007/978-3-540-74450-4_1
- N. Karmarkar and R. M. Karp. *An Efficient Approximation Scheme for the One-Dimensional Bin-Packing Problem*. FOCS 1982, pp. 312–320. doi.org/10.1109/SFCS.1982.61
- W. Fernandez de la Vega and G. S. Lueker. *Bin Packing Can Be Solved Within $1 + \varepsilon$ in Linear Time*. Combinatorica 1(4):349–355, 1981. doi.org/10.1007/BF02579456
- J. Balogh, J. Békési, G. Dósa, L. Epstein, A. Levin. *A New Lower Bound for Classic Online Bin Packing*. Algorithmica, 2021. arxiv.org/abs/1807.05554 — the 1.54278 online lower bound.
- J. Balogh, J. Békési, G. Dósa, L. Epstein, A. Levin. *A New and Improved Algorithm for Online Bin Packing*. 2018. researchgate.net/publication/318260129 — the 1.57829 Advanced Harmonic upper bound.
- C. C. Lee and D. T. Lee. *A Simple On-Line Bin-Packing Algorithm*. Journal of the ACM, 1985 — the classical Harmonic algorithm's ≈ 1.691 ratio.
- [Bin packing problem](#) — Wikipedia
- [3-partition problem](#) — Wikipedia
- [Codeforces Gym 102770B](#) — Bin Packing Problem
- [Codeforces 1066D](#) — Boxes Packing
- [Codeforces 2038L](#) — Bridge Renovation

Read online, with runnable code: <https://priyanshudebnath.me/blog/bin-packing-np-hardness-approximation-algorithms/>